Campus quick-commerce build playbook

Oct 6, 2026 · @ujjwal

Build it, as a small well-run store with the app as the easy half. Three residential campuses outside every delivery zone, with late-night demand and nothing serving it, is a real gap. The software is roughly ten working sessions in Antigravity; stock, hours, permission and delivery fees decide whether it makes money.

Decisions made for you

Each is a default. Tell me which you would change and I will rework the prompts it touches.

Decision	Choice	Why
Form	Mobile-first web app, installable to the home screen	A link in hostel WhatsApp groups beats an app-store listing. No review, no store fees.
Stock	Your own stock, about 150 to 200 fast-moving items	Accurate availability, real 15-minute delivery, and you stay a small retailer instead of a marketplace operator.
Delivery	A fixed list of hostels and gates across the three campuses	No maps, no GPS, no address parsing.
Hours	One evening-to-late-night window	Campus shops are shut then, and one shift can cover it.
Login	Google sign-in, asked only at checkout	Free and verified. SMS codes cost money per message and need telecom registration.
Payment	Pay on delivery, by UPI or cash	No gateway, no KYC wait, no refunds to handle while you learn the demand.

On "same-to-same Blinkit": the prompts reproduce the structure every quick-commerce app shares, under your own name, colours and photos. That means a category grid, add-to-cart steppers, a sticky cart bar and instant search. Blinkit's branding and product images stay out. Copying them invites a takedown, and a look-alike reads as a knock-off, which is the "local app" impression you want to avoid.

What you have not thought about

The app does not decide this business. These ten points do, in the order they bite.

1. **You cannot mark up.** MRP caps your price, and branded packaged goods leave a retailer thin margins. Your levers are the delivery fee, a minimum order and higher-margin lines, so all three are admin settings from day one.
2. **Know your break-even before buying stock.** Illustration only; replace every number with your own quotes. A ₹200 order at 15% margin earns ₹30, plus a ₹15 delivery fee, minus ₹15 to the runner: ₹30 an order. At ₹10,000 of monthly fixed cost you break even at 334 orders a month, about 11 a night. Forty a night leaves about ₹26,000 a month, which is good student income and not a venture-scale business.
3. **Test demand by hand this week, while you build.** Run a WhatsApp catalogue of 30 items with a shared sheet for three evenings. You get the real basket, the real peak hour, and whether people pay a delivery fee. If manual ordering cannot reach double-digit orders a night, software will not fix it.
4. **Get permission before stock.** A store run from a hostel room, and runners entering three campuses, both need a yes from the people who run them. Get it in writing before you spend on inventory. "Deliver to the main gate" is the fallback that avoids the entry question.
5. **Licences.** I am not a lawyer, so confirm each point before launch. Selling food needs an FSSAI registration; since 1 April 2026 the basic tier covers turnover up to ₹1.5 crore, at ₹100 a year (Chambers, ComplyLocal). Two sources say a business selling its own food on its own site does not need the heavier e-commerce licence (LexComply, ComplyLocal). Both write about restaurants and home kitchens, not resellers, and other compliance blogs claim the opposite. Settle it on the FSSAI portal or with a local consultant. For GST, selling your own goods on your own site keeps you on the normal turnover threshold, while running a marketplace for other shops forces registration (TaxBuddy).
6. **What you cannot sell.** Cigarettes will be the most-requested item and the answer is no: tobacco sales within 100 yards of an educational institution are prohibited (PIB). No medicines without a drug licence, and no alcohol.
7. **A browser tab cannot wake a locked phone.** If new orders ring only inside the admin page, you will miss orders. Prompt 10 sends every order to a Telegram group, which is free and works on a locked phone.
8. **Catalogue entry is the real grind, and photos decide the look.** Two hundred products each need a name, size, MRP, price, stock count, photo and search keywords. Start the sheet today from the attached template. Shoot your own photos on one white background with one framing; uniform photos do more for "not local" than any styling.

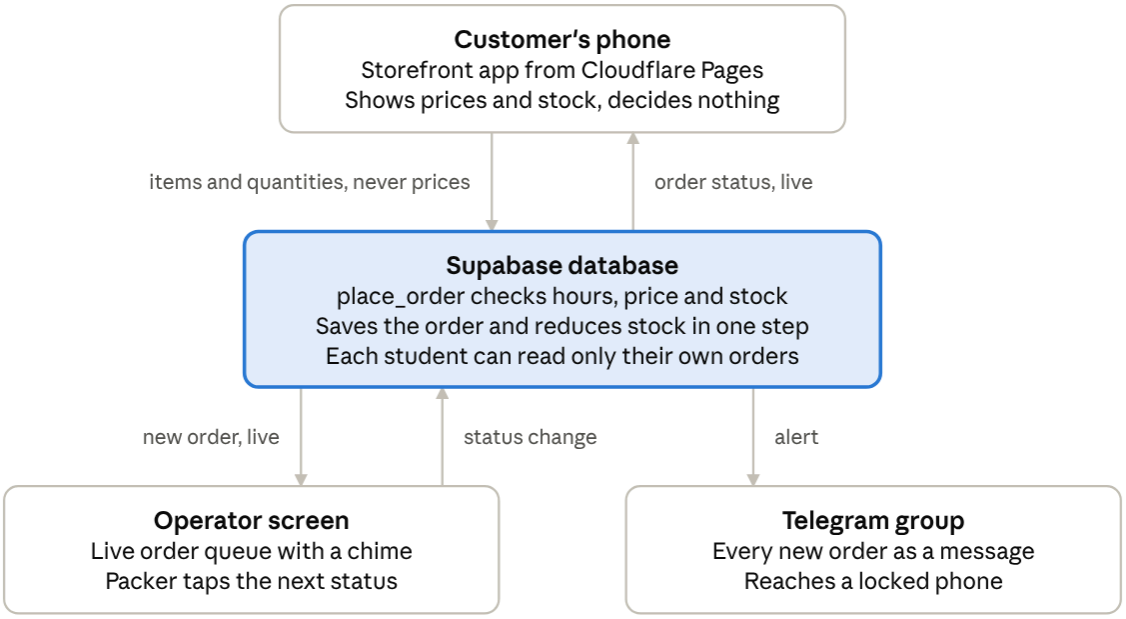
9. **Breaks and exams.** Your market empties in the vacations, so buy short-dated stock accordingly and plan to pause. Exams hit your own availability too. One runner per campus, paid per delivery, covers the nights you cannot and solves gate entry.
10. **A national player can switch this area on any day.** Your edge is being inside the gate, open later, and stocked for exactly this crowd. Software speed is not the moat.

You will also hold names, phone numbers and room numbers. Prompts 3 and 4 exist so one student can never read another's order.

Scope and stack

v1 is one store, one catalogue, one checkout and one operator console, with no server of your own. A static app talks straight to a managed database, and the database enforces every rule.

The phone asks, the database decides



order flow · 4 parts

An order request carries only items and quantities. The database sets the price, checks stock, then tells the operator screen and Telegram.

Area	In v1	Left out on purpose
Browsing	Home with category grid and product rails, category pages, product sheet	Ratings, reviews, recommendations

Area	In v1	Left out on purpose
Search	Instant and typo-tolerant, with similar items and a logged dead end	Voice search
Checkout	Cart with bill breakdown, Google sign-in, saved hostel and room, pay on delivery	Payment gateway, coupons, wallet, scheduled slots
Delivery	Fixed delivery points with their own ETA and fee, live status timeline	Maps, GPS, rider app, live rider tracking
Operator console	Live order queue with sound, pack list, stock control, product editor with CSV import, settings, daily numbers	Multiple stores or vendors, staff roles
Reach	Installable web app, Telegram alerts to you	SMS, push notifications, native apps

Everything in the right-hand column adds weeks, and none of it is needed to learn whether people order.

How search behaves

Search runs in the browser over the whole catalogue, so results appear as the customer types. It tolerates typos and aliases such as "magi", "biskit" or "cold drink". An out-of-stock match shows similar items that are in stock. A search with no match shows the nearest category's bestsellers and a "Request this item" button, and the query is logged. That log is your buying list.

Stack

Layer	Choice	Free-plan fact that matters
App	React, Vite, TypeScript, Tailwind, shadcn/ui	The same family as your BacktestLab front end.
Backend	Supabase: Postgres, Google sign-in, realtime, storage, functions	500 MB database, 1 GB storage, 5 GB egress, 2 active projects, paused after a week idle, no backups. Pro is $25 a month.
Search	MiniSearch, in the browser	No server calls at 200 products.

Layer	Choice	Free-plan fact that matters
Hosting	Cloudflare Pages	500 builds a month and custom domains on the free plan.
Order alerts	Telegram bot through a Supabase function	Free.
Payments	None in v1	Add a gateway when volume justifies the paperwork.

Three rough edges of the free plan:

- Google's sign-in screen shows a supabase.co address, not your domain. Fixing it needs Pro plus a $10 a month add-on.
- There are no backups. Prompt 9 adds an export button; use it weekly and move to Pro once revenue covers it.
- You get two active projects. If BacktestLab already holds one, this is your last.

Models: which one for which job

Make Gemini 3.8 Flash the builder and use the other models only to check its work. Your screenshot shows Gemini 3.1 Pro highlighted; if that is your default, change it.

Model in your picker	Job in this build	Setting
Gemini 3.8 Flash	Builds every feature prompt	Medium by default, High for Prompts 3, 7 and 8
Gemini 3.1 Pro	Challenges the spec, builds the first look, takes bugs Flash failed twice, audits after 2 November	High
Claude Opus 4.6 (Thinking)	Independent attack test and pre-launch audit, until 2 November	As listed
Claude Sonnet 4.6 (Thinking)	Second build of the look, for comparison, until 2 November	As listed
Gemini 3.7 Flash and 3.6 Flash	Lighter on the shared Gemini allowance; use them for copy and small style fixes when it runs low	Medium

Skip GPT-OSS 120B. It is an older open-weights model and it leaves on 2 November.

Why this split

- **3.8 Flash builds.** Google now runs its managed Antigravity agent and the Antigravity SDK on it by default (Gemini API docs). On independently run coding tests it is level with or ahead of 3.1 Pro: 80.0% against 78.8% on SWE-bench, and 81.3% against 70.8% on Terminal-Bench 2.1 (BenchLM, Vals AI runs).
- **3.1 Pro gets the first look.** Google pitches Pro at large design systems and strict design rules, and one reviewer argues Flash models trade depth for speed on polished multi-screen design (UXMagic). That is positioning and opinion, not a test, so Prompt 2 builds the look twice and you judge.
- **A different model checks.** A model reviewing its own work shares its blind spots. A reviewer from another family is the cheapest verification you have.
- **Medium, not High, by default.** Google documents Medium as the default level for 3.8 Flash. High spends more of a weekly allowance, so keep it for the database, checkout and operator console.

The Notice badge: you have four weeks of Claude

Claude Sonnet 4.6, Claude Opus 4.6 and GPT-OSS 120B leave Antigravity on 2 November 2026 (Antigravity models). Their replacements, Claude Sonnet 5.5 and Opus 5.5, are limited to paid non-trial Pro and Ultra plans, and they are not in your picker. So run Prompts 2, 4 and 12 on Claude before that date. After it, Gemini 3.1 Pro takes the reviewer role and nothing else in this plan changes.

Allowance

Gemini models draw on one allowance; Claude and GPT draw on a separate one. Off the paid plans both refresh weekly (Antigravity plans), so the Claude reviews cost nothing from your building budget. "View usage" in the picker shows both. If the weekly limit becomes your bottleneck, Google's student offer, open until 31 December 2026, prices AI Pro at ₹489 a month (Business Today), and Pro refreshes every five hours. `/boost` and `/teamwork-preview` are paid-plan features this build does not need.

What I cannot vouch for

The 3.6 to 3.8 Flash models are newer than my training data. This split rests on Google's documentation and public benchmarks read on 6 October 2026, not on running these models against your code. Benchmarks also do not measure taste, which is why Prompt 2 builds the look on two models. If a model fails the same task twice, switch model instead of re-prompting.

How to run Antigravity for this build

Give the agent its rules in files, hand it one small task at a time, and treat "done" as a claim until you have checked it on your phone.

Settings, once

1. **Review policy.** In Settings, Agent tab, set the artifact review policy to "Request review". Plans then stop for your approval (docs).
2. **Permissions.** Keep the sandboxed preset: "Default" on macOS and Linux, "Proceed in Sandbox" on Windows. Never "Turbo" or "Always Proceed" (docs).
3. **Database connection.** In Settings, Customizations, Installed MCP Servers, add Supabase from the MCP Store (docs). Use the project-scoped address from the MCP tab of your Supabase dashboard. The agent can then apply database changes itself. If the connection fails, it writes the SQL files and you paste them into the Supabase SQL editor. Switch it to read-only on launch day; the Supabase server supports `read_only=true`.

Menu names follow the current Antigravity docs. If yours differ, search Settings for the same words.

Starter files

Five files are attached in the chat. The spec was sent twice, so use the later copy. Put `AGENTS.md` in the project root and the other four in a `docs/` folder. Read `SPEC.md` once and correct anything that is wrong, because it is the one place your decisions live.

File	What it does
`AGENTS.md`	Rules the agent reads on every task. Antigravity loads a root `AGENTS.md` automatically (docs).
`docs/SPEC.md`	The product contract: scope, data, order rules, search, security. Every prompt points at it.
`docs/DESIGN.md`	Colours, type, spacing, components, and the quality bar for "not local".
`docs/PROGRESS.md`	The agent's log. Each new conversation reads it, so nothing depends on chat history.
`docs/catalog-template.csv`	The sheet to start filling today. Sample rows are placeholders; copy sizes and MRPs from the packs.

The loop for every prompt

1. Start a new conversation and pick the model named on the prompt.
2. Run every prompt in Planning mode, or begin the message with `/plan` (docs). Fast mode is for one-line fixes. Where a prompt says "Plan: read closely", check the plan against the spec before you proceed. Elsewhere a skim is enough.
3. When the agent says it is done, run the "You verify" checks yourself, on your phone where it says so.
4. If a check fails, send one problem per message: a screenshot, what you expected, what happened.
5. After two failed fixes in a row, stop. Open a new conversation on a different model and say what was tried.
6. When the checks pass, commit to git. Each commit is your undo button.

Sessions

Every stopping point leaves something that works. If the weekly allowance runs out, you lose time, not work.

Session	Prompts	You end with
1	Setup, 1, 2	The look, live on your phone
2	3, 4	A database that passes an attack test
3	5	A browsable store with a working cart
4	6	Search
5 and 6	7	Real orders, end to end
7	8	The operator console
8	9, 10	Catalogue tools and Telegram alerts
9	11	An installable, fast app
10	12	An audit and its fixes

In the same first week, outside Antigravity: the manual demand test, the catalogue sheet, the photos and the permission letter.

The prompts

Twelve prompts, in order, each pasted as written into a new conversation. Your details live in the starter files, so the prompts need no editing. Each one ends with checks that you run yourself.

Before Prompt 1

1. Create an empty project folder and open it in Antigravity. Add the five starter files. Replace `[BRAND]` with your name once you have one.
2. Create a private GitHub repository for the folder.
3. Create a Supabase project in the Mumbai region. Keep its project URL, publishable key and secret key at hand.
4. Connect the Supabase server to Antigravity, as in the settings above.
5. Create a Cloudflare account. You connect it after Prompt 2.

Prompt 1: Challenge the spec

Model: Gemini 3.1 Pro, High · No code is written.

```
Read AGENTS.md, docs/SPEC.md and docs/DESIGN.md. Do not write code or create
files in this task.

Act as a sceptical senior engineer reviewing this spec before a build starts.
Give me:

1. Contradictions and gaps: anything two sections disagree on, or a decision
   the build will need that the spec does not make.
2. The five most likely ways this app fails in real use by students on phones
   at night, and whether the spec already prevents each one.
3. Anything in the stack or data model you would change for a store with
   about 200 products and 50 orders a day, and why. Do not propose
   infrastructure for scale we do not have.
4. Questions for me, numbered, each answerable in one line.

Be direct. Do not praise the spec. If something is fine, do not mention it.
```

You verify

- You get gaps and questions, not code. Answer the questions in one message.
- Then send: "Update docs/SPEC.md with my answers and show me the exact lines you changed." Read those lines.

Prompt 2: Project, design system and the look

Model: Gemini 3.1 Pro, High, then a second run on Claude Sonnet 4.6 (Thinking) · **Plan:** skim

Before. Ask the agent: "Commit the starter files, then create and switch to a branch named look-a."

```
Set up the project and build the design system. No database and no real data
in this task.

1. Scaffold the app with the stack in AGENTS.md. Add scripts: dev, build,
   preview, typecheck, lint. Add .env.example with VITE_SUPABASE_URL,
   VITE_SUPABASE_ANON_KEY and SUPABASE_SECRET_KEY (names only, no values).
   Make sure .env.local is git-ignored.
2. Implement the tokens in docs/DESIGN.md as CSS variables wired into
   Tailwind. Self-host the font.
3. Build every component listed in docs/DESIGN.md exactly as described, with
   typed props and mock data only.
4. Create a /styleguide route that shows every component in every state
   (default, in cart, low stock, out of stock, long name, no image, loading).
   Below that, one fully composed mock Home screen and one mock Category
   screen using about 12 realistic sample products.
5. App shell: routes for /, /c/:slug, /search, /cart, /checkout, /orders,
   /orders/:id, /account and /admin as placeholders, a 404 page, and the
   centred 480 px column layout.
6. Make deep links and page reloads work on Cloudflare Pages
   (single-page-app fallback).
7. With the image tool, generate one placeholder set in a single consistent
   style: 8 category tile illustrations on the tint colours from DESIGN.md,
   and 12 generic, unbranded product images on white. No real brand names,
   logos, mascots or pack artwork.

Before you finish: run typecheck, lint and build. Open /styleguide at 390 px
wide, take screenshots, and check them against the quality bar in DESIGN.md.
Fix what fails, then list anything that still does not meet it.
```

The second run. Ask the agent to commit, then "create and switch to a branch named look-b from main". Open a new conversation on Claude Sonnet 4.6 (Thinking) and paste the same prompt. You now have two versions of the look, built from identical instructions.

You verify

- Connect the GitHub repository in Cloudflare Pages: build command `npm run build`, output folder `dist`. Each branch gets its own preview address. The main branch fails to build until you merge, which is expected.

- Open `/styleguide` from both branches on your own phone. Check with your thumb: the ADD button is easy to hit, nothing moves as images load, long names stop at two lines, out of stock is obvious.
- Keep the better branch: "Merge look-a into main and delete look-b", or the reverse.
- Improve only the styleguide until you like it, one change per message with a screenshot. Every later screen is built from these components, so this is the cheapest place to fix the look.

Prompt 3: Database, rules and sample data

Model: Gemini 3.8 Flash, High · **Plan:** read closely

```
Build the database for docs/SPEC.md. Plan first and wait for my approval
before applying anything.

Deliver as SQL migrations in supabase/migrations/ and apply them to the
connected Supabase project:

1. Every table in "Data model" with its constraints, foreign keys and useful
   indexes.
2. Row Level Security on every table, following "Security" exactly.
3. Functions:
   - is_admin()
   - place_order(client_request_id, items [{product_id, qty}],
     delivery_point_id, room, substitution_pref, note), implementing every
     rule in "Order rules"
   - cancel_my_order(order_id)
   - admin_set_order_status(order_id, new_status, reason), enforcing
     "Status flow" and restoring stock on cancel
   - admin_adjust_stock(product_id, delta)
   - admin_set_payment(order_id, status, paid_via)
   - admin_remove_order_item(order_item_id), as described in "Status
     flow"
4. Realtime enabled on orders only.
5. Storage bucket product-images: public read, admin-only write.
6. Seed data, clearly marked as sample: 8 categories, 40 realistic products
   spread across them, 10 delivery points across three campuses with
   placeholder names, and the settings row with the launch defaults, except
   opening hours set to all day so we can test at any time.
7. src/lib/supabase.ts and generated TypeScript types for the schema.

In the plan include one table: for each database table, who can select,
```

```
insert, update and delete, and through what (policy or function). I will
review that table before approving.

Do not build any UI in this task.
```

You verify

- Before approving, read the access table in the plan against section 11 of `SPEC.md`. The rows for `admins`, `orders` and `order_items` must show no direct client writes.
- In the Supabase dashboard, every table shows Row Level Security enabled and the Security Advisor lists no errors.
- Put the project URL and both keys in `.env.local`. Add the two `VITE_` values to the Cloudflare Pages project settings.
- Do not trust this step on sight. Prompt 4 tests it.

Prompt 4: Attack test by a different model

Model: Claude Opus 4.6 (Thinking) until 2 November, then Gemini 3.1 Pro, High · **Plan:** skim

```
You did not write this database. Your job is to break it.

Read "Order rules", "Status flow" and "Security" in docs/SPEC.md, and every
file in supabase/migrations/. Then:

1. Review. List every way a malicious or careless client could: pay less
   than the real price; order more than the stock; read or change another
   user's data; make themselves admin; order while the store is closed;
   create duplicate orders; corrupt stock through cancel and re-order
   sequences. For each, say whether the current SQL prevents it and quote
   the line that does.

2. Prove it. Write scripts/security-test.mjs that runs against the real
   project. It may use SUPABASE_SECRET_KEY from .env.local only for set-up
   and clean-up: creating and deleting its own test users, category and
   products, and temporarily changing settings, which it restores. Every
   test action runs as a normal signed-in user with the publishable key.
   Cover at least:
   - a tampered price is ignored
   - quantity above stock, above max_per_order, zero and negative: rejected
   - an order below the minimum: rejected
   - store closed: rejected
   - inactive product: rejected
   - user A cannot read user B's order or profile
   - a non-admin cannot write products, settings, orders, order_items or
     admins, and cannot call any admin function
```

```
   - the same client_request_id twice creates one order
   - two simultaneous orders for the last unit: exactly one succeeds and
     stock ends at 0
   - cancelling restores stock exactly once, even if called twice
   - removing an item recalculates the total, never raises the delivery
     fee, and sets that product's stock to 0
   The script deletes everything it created, so it is safe to run again.

3. Add "npm run test:security". It prints one PASS or FAIL line per case and
   exits non-zero on any failure.

4. Run it. For every FAIL, fix the SQL in a NEW migration, run again, and
   show me the final output.

Report failures first. Never describe a failing run as a success.
```

You verify

- Run `npm run test:security` yourself in the terminal. Every line says PASS.
- Then prove the test can fail. Ask: "In a new migration, remove the stock check from place_order, run the tests, show me the failures, then restore it with another migration and run again." A test that cannot go red proves nothing.
- After launch, run it only while the store is closed, because it briefly changes settings.

Prompt 5: Storefront and cart

Model: Gemini 3.8 Flash, Medium · **Plan:** skim

```
Build the customer storefront on real data, using only the components from
/styleguide.

1. Catalogue loading as described in SPEC "Catalogue loading and similar
   items".
2. Home: a header with the open or closed state from settings and the chosen
   delivery point with its ETA (tap to change; remembered on the device
   before sign-in); the search field, which opens /search; the category
   grid; and product rails driven by the rail field.
3. Category page /c/:slug: subcategory chips when present, a two-column
   grid, in-stock items first.
4. Product sheet: everything in SPEC scope item 3, with the AddStepper and
   "Similar items".
5. Cart: a persisted store keyed by product id. The AddStepper behaves
   identically on cards, rails, the sheet and the cart page. It respects
   stock_qty and max_per_order, with a toast when capped. The CartBar shows
   whenever the cart has items.
6. /cart: line items and the BillSummary computed from settings, with the
```

```
   minimum-order and free-delivery nudges and the savings line. Label the
   total as an estimate confirmed at checkout. Proceed leads to a
   placeholder for now.
7. When the store is closed, browsing and the cart still work, a banner
   says when it opens, and Proceed is disabled.
8. Reconcile the cart after every catalogue refresh, as the SPEC describes.

Verify in the browser at 390 px: add from a rail; change the quantity in the
sheet and confirm the card and CartBar agree; reload and confirm the cart
survived; set one product's stock to 0 in the database and confirm the
storefront and cart correct themselves within a minute. Record the
walkthrough.
```

You verify, on your phone at the Cloudflare address

- Repeat those four actions yourself.
- Reload on mobile data. The home screen appears at once from the saved catalogue.
- Set `store_open` to false in the Supabase table editor. The banner appears and Proceed is disabled.

Prompt 6: Search

Model: Gemini 3.8 Flash, Medium · **Plan:** skim

```
Build search exactly as specified in SPEC "Search".

1. A MiniSearch index built from the cached catalogue with the fields and
   weights in the spec, rebuilt when the catalogue changes. Prefix and fuzzy
   matching. Normalise queries: lowercase, trimmed, punctuation and repeated
   spaces removed.
2. /search: the field focused on arrival, results as the user types, the
   three result tiers in order, the "Similar in stock" row under
   out-of-stock matches, and the no-match screen with nearest-category
   bestsellers and "Request this item".
3. The empty search screen described in the spec.
4. Logging to search_misses and item_requests under the spec's rules. Never
   on every keystroke.
5. Add 3 to 6 alias keywords to every seed product (generic names, common
   misspellings, Hinglish) so the behaviour can be tested.
6. Unit tests for the search function on a fixed fixture catalogue: exact,
   prefix, one typo, two typos, alias, out of stock with alternatives, and
   no match. Wire them into "npm test".

Then give me a table of 20 queries to type on my phone, with the top three
results you expect for each on the seed data.
```

You verify

- Type the 20 queries on your phone. Send back any that disagree with the table.
- Type 10 more the way your friends would: misspelt, half-typed, in Hinglish.
- Search for something you do not stock, wait two seconds, and check that a row appears in `search_misses`.

Prompt 7: Sign-in, checkout and order tracking

Model: Gemini 3.8 Flash, High · **Plan:** read closely

```
Build sign-in, checkout and order tracking. Plan first and wait for
approval. Begin the plan with the exact dashboard steps I must do myself:
the Google OAuth client, the Supabase Google provider, and the redirect
addresses for local and Cloudflare.

1. Sign-in: Google through Supabase, asked only when the user taps Proceed
   in the cart or opens Orders. Afterwards return them to where they were
   with the cart intact. Detect in-app browsers where Google sign-in is
   blocked (Instagram, Facebook and similar) and show "Open in Chrome or
   Safari" with a copy-link button instead of a broken login.
2. Profile: a one-time form for name, 10-digit Indian mobile number,
   delivery point and room. Editable later from /account, which also has
   sign-out.
3. /checkout: address summary (editable), substitution preference, optional
   note, final bill, the line "Pay on delivery: UPI or cash", and Place
   order.
4. Placing: call place_order with a client_request_id created when the
   checkout screen opens. Disable the button while pending. A retry after a
   network failure reuses the same id. On success clear the cart and open
   /orders/:id. On a structured error (out of stock, price changed, store
   closed, below minimum, too many active orders) correct the cart and
   explain in one sentence what changed.
5. /orders/:id: the StatusTimeline updating live through Realtime, with
   polling as a fallback; order number, items and bill; Cancel only while
   the status is placed; "Need help?" opening WhatsApp to the support phone
   with the order number filled in.
6. /orders: history, newest first, with "Order again" that adds the items
   still available.
7. The four static pages in SPEC scope item 9, linked from the footer and
   from /account. Write short plain drafts, mark each "DRAFT: owner to
   review", and leave clear placeholders for business name, address, phone
   and food-licence number.
```

```
Verify end to end in the browser: sign in, complete the profile, place an
order, change its status in the database and watch the timeline move, cancel
a second order and confirm the stock returned. Then go offline during
placement and confirm a retry creates no duplicate order.
```

You verify

- In Google Cloud, set the sign-in consent screen to "In production". In "Testing" only listed test users can sign in.
- Place a real order from your phone. In the Supabase table editor, see the order, its items, and the stock reduced.
- Tap Place order twice quickly, and once in airplane mode. You get one order, never two.
- Open the store link from Instagram. You see the "open in browser" message, not an error.
- Run `npm run test:security` again. Still all PASS.

Prompt 8: Operator console, orders and stock

Model: Gemini 3.8 Flash, High · **Plan:** read closely

Before. Sign in once on the site, then make yourself an admin in the Supabase SQL editor:

```
insert into admins (user_id)
select id from auth.users where email = 'your-google-address@example.com';
```

```
Build the operator console at /admin for admins only. Non-admins get the 404
page, and the database refuses them whatever the interface shows.

1. Orders, the default tab: a live queue through Realtime, grouped Placed,
   Accepted, Packed, Out for delivery, then today's Delivered and Cancelled.
   Order cards and timing colours as in SPEC "Operator console details". One
   large button per card for the next status. Cancel asks for a reason.
   "Remove item" on any line until the order is packed. Tap-to-call and
   WhatsApp links. "Mark paid: UPI / Cash".
2. Alerts: the "I'm online" switch described in the spec, with the repeating
   chime, flashing tab title, screen wake lock, the visible warning when
   sound is blocked or the live connection drops, and automatic reconnect.
3. Pack view: a full-screen checklist of one order's items in large text.
4. Stock tab: instant search, large minus and plus buttons, one-tap "Out of
   stock", a low-stock filter (3 or fewer), last-updated time. Every change
   goes through admin_adjust_stock.
5. A store switch in the header: Open or Closed, with today's hours beside
   it.
6. Usable on a 390 px phone for the packer and runner, and on a laptop.
7. Load /admin as a separate chunk that customers never download.
```

```
Verify with two browser windows: place an order as a customer in one and
confirm the chime and card appear in the other within two seconds; walk the
order to Delivered and confirm the customer's timeline followed every step.
Then sign in as a non-admin and confirm /admin and every admin function are
refused.
```

You verify

- Use two real devices: order on one, operate on the other. Time how long the card takes to appear.
- Lock the operator phone and place another order. You will probably not hear it, which is what Prompt 10 fixes.
- Set one product's stock to 1, then order it from two phones at once. One succeeds.
- Remove an item from a test order. The total drops, the delivery fee does not rise, and that product shows out of stock.

Prompt 9: Catalogue, settings and daily numbers

Model: Gemini 3.8 Flash, Medium · **Plan:** skim

```
Add the remaining operator tabs.

1. Products: a list with search and category filter; an add and edit form
   for every product field; photo upload from camera or gallery that crops
   to square, resizes to 600 px and converts to WebP under about 40 KB in
   the browser before upload, stored with long-lived cache headers.
   Deactivate instead of delete.
2. CSV import matching docs/catalog-template.csv: a preview with per-row
   validation errors before anything is written; upsert by sku; a summary of
   created, updated and skipped rows. A bulk photo upload that matches files
   to products by image_filename. CSV export of products, and of orders for
   a date range.
3. Categories: add, edit, reorder, tile image upload.
4. Settings: hours (including closing after midnight), minimum order,
   delivery fee, free-delivery threshold, active-order limit, support phone,
   notice text; and delivery points (add, rename, ETA, fee override,
   activate or deactivate).
5. Today, on the business day defined in the spec: orders, revenue with
   the UPI and cash split, average order value, cancellations, top 10 items,
   orders per hour, out-of-stock products, the 20 most frequent failed
   searches of the last 7 days, and item requests.
```

```
Verify: import the template with one deliberately broken row and confirm it
is reported and nothing partial is written. Upload a 5 MB phone photo and
report the stored file size.
```

You verify

- Import your real catalogue sheet. Fix the rows it rejects.
- Bulk-upload your photos and scroll the storefront on your phone. Every card has an image with the same framing.
- Change the delivery fee in Settings and confirm a new order uses it.

Prompt 10: Order alerts on Telegram

Model: Gemini 3.8 Flash, Medium · **Plan:** skim

Before. In Telegram, create a bot with @BotFather, create a group for your team, and add the bot to it.

```
Send every new order to our Telegram group so we are alerted even when the
operator screen is closed.

1. A Supabase Edge Function named notify-order. Given an order id, it loads
   the order and items with the service role and posts a compact message:
   order number, delivery point and room, customer name and phone, items
   with quantities, total, substitution preference, note, and a link to
   /admin.
2. Trigger it from the database on every new order, so it fires however the
   order was created. It must never block or fail order placement: errors
   are logged, not raised.
3. Also notify when a customer cancels.
4. The bot token and chat id are function secrets, never in the repository.
   Give me the exact steps to set them and to find the chat id.

Verify by placing a test order and showing the function's log line and
response status.
```

You verify

- Lock your phone and order from another phone. The Telegram message arrives within a few seconds.
- Search the repository on GitHub for the first characters of the bot token. There are no results.

Prompt 11: Installable, fast, polished

Model: Gemini 3.8 Flash, Medium · **Plan:** skim

```
Make it feel like an app and make it fast. Do not change behaviour or add
features in this task.

1. PWA: a manifest with our name, theme colour and an original icon set
   drawn from the logo in DESIGN.md; installable on Android and iOS; an
   offline app shell with a friendly offline screen. The service worker must
   never serve stale prices or stock. Show "Update available, tap to
   refresh" when a new version is deployed.
2. Speed on a mid-range Android phone on 4G: route-level code splitting,
   lazy images with fixed aspect ratios, font preloading, no layout shift.
   Run Lighthouse in mobile mode on / and on a category page. Report scores
   before and after. Aim for 90 or more on performance and 95 or more on
   accessibility.
3. A polish pass against the DESIGN.md quality bar on every customer screen
   at 360, 390 and 430 px. List each issue you found and fixed.
4. Sharing: title, description and an Open Graph image, so the link looks
   right when pasted into WhatsApp.
5. Error capture: record unhandled errors and failed order placements in
   client_errors, per the spec, with no personal data in the message.
```

You verify

- Install it to your home screen on Android and on an iPhone. It opens full screen with your icon. Sign in and place an order from the installed app on both.
- Paste the link into a WhatsApp chat. The preview shows your name and image.
- Turn on airplane mode and open the app. You see the offline screen, not a browser error.
- Run Lighthouse yourself in Chrome on the live address and compare with the agent's numbers.

Prompt 12: Pre-launch audit by a different model

Model: Claude Opus 4.6 (Thinking) before 2 November, otherwise Gemini 3.1 Pro, High · **Plan:** skim

```
You are auditing this app the day before real customers and real money. You
did not build it. Assume the builder's notes in docs/PROGRESS.md may be
wrong and check the code.

1. Run npm run test:security, npm test, typecheck, lint and build. Report
   the raw results.
```

```
2. Trace one order through the code from the customer's tap to Delivered.
   List every place it could silently fail or mislead the customer or the
   operator.
3. Go through SPEC v1 scope item by item: Done, Partly or Missing, with the
   file that proves it.
4. Go through the "Not in v1" list and flag anything built or half-built
   anyway.
5. Search the repository and the built dist/ folder for secrets, the
   service-role key, test credentials, console logs containing personal
   data, and sample data that must not ship.
6. Review Row Level Security against the live database, not only the
   migration files.
7. Give me the ten biggest problems ranked by damage times likelihood, each
   with a one-line fix. Fix nothing yet.

Lead with the problems. No reassurance.
```

You verify

- Read the ten problems. Anything touching money, stock or personal data gets fixed.
- Fix them in a new conversation on Gemini 3.8 Flash, one problem per message.
- Run this audit once more. Stop when a run finds nothing that touches money, stock or personal data.

Launch checklist

Launch when every box is ticked, not when the app looks finished.

- [ ] Sample products, test orders and test users are deleted, and the real catalogue is imported with counted stock.
- [ ] Delivery points, hours, fees, minimum order and support phone are set in Settings.
- [ ] You and your packer are in `admins`.
- [ ] Google's consent screen is "In production", and the live address is in Supabase's redirect list. In Supabase Authentication, email confirmation is on and anonymous sign-in is off.
- [ ] The Supabase connection in Antigravity is read-only or removed.
- [ ] A short domain of your own is connected in Cloudflare Pages.
- [ ] The four footer pages carry your real business details and licence number.
- [ ] A Telegram alert reaches a locked phone.
- [ ] Ten friends order at the same time on hostel Wi-Fi, and you time each order from tap to door.

- [ ] The site states a fallback: if the app is down, order by WhatsApp to the support number.

The first two weeks

Change one thing at a time and let these numbers decide. Repeat orders come from the orders export; the rest are on the Today screen.

Number	What it decides
Orders per night	Whether you are above break-even
Average order value	Whether the delivery fee and minimum order are right
Customers who order again within 7 days	Whether the demand is real
Minutes from tap to door	Whether you keep the promise you sell
Cancellations, with reasons	Stock accuracy and prank orders
Failed searches and item requests	What to stock next
Items that ran out	What to buy more of

What to add next, only when the numbers ask

1. Prepaid UPI through a payment gateway, when unpaid or prank orders cost more than the paperwork.
2. Order-status messages to customers, when "where is my order" calls pile up.
3. A first-order offer, when sign-ups stall.
4. Combos, when the average order sits under your free-delivery line.
5. Supabase Pro for backups, as soon as revenue covers $25 a month.

Sources

All read on 6 October 2026.

- Antigravity: models and plan availability, plans and allowance, rules files, /plan, /boost, artifact review, agent settings, MCP servers, Gemini 3.8 Flash announcement
- Models: Gemini API docs on 3.8 Flash, BenchLM comparison of 3.1 Pro and 3.8 Flash, UXMagic on models for interface design
- Services: Supabase pricing, Supabase MCP server, Cloudflare Pages limits

- India: Chambers on FSSAI thresholds, ComplyLocal on basic registration, LexComply on selling online, TaxBuddy on GST registration, PIB on tobacco sales, Business Today on the student offer

APPENDIX A: AGENTS.MD (RULES FILE)

[BRAND] — rules for every task

Read `docs/SPEC.md`, `docs/DESIGN.md` and `docs/PROGRESS.md` before planning anything. They are the source of truth. If a request conflicts with them, stop and ask the owner.

The product in one line

A single-store quick-commerce web app for three neighbouring campuses in Bhopal: students order snacks and essentials on their phones, and the store's own stock is delivered to a fixed list of hostels and gates.

Stack (do not change without asking)

- Vite + React + TypeScript, React Router, Tailwind CSS, shadcn/ui primitives, lucide-react icons.
- TanStack Query for server data. Zustand (persisted) for the cart.
- Supabase: Postgres, Auth (Google), Realtime, Storage, Edge Functions. Region: Mumbai.
- MiniSearch for product search in the browser. vite-plugin-pwa for install and the offline shell.
- Hosting: Cloudflare Pages, static build in `dist/`. There is no custom backend server.
- Use the current stable version of each. Do not mix configuration styles from different major versions.

Rules that are never broken

1. **The browser is untrusted.** Prices, fees, totals, stock and order status are computed and enforced in Postgres (functions + Row Level Security). The client never inserts into `orders` or `order_items` directly and never sends a price the server trusts.
2. **Row Level Security is on for every table**, with explicit policies. No table is left open "for now".
3. **Only the Supabase publishable (anon) key appears in client code.** Never the secret / service-role key. Secrets live in `.env.local` (git-ignored) or in Supabase function secrets.
4. **Every schema change is a new timestamped SQL file** in `supabase/migrations/`. Never edit a migration that has already been applied.
5. **Money is whole rupees stored as integers.** Times are stored in UTC and shown in Asia/Kolkata.
6. **Storefront UI uses only the tokens and components in `docs/DESIGN.md`.** No one-off colours, font sizes, shadows or spacing. No emoji as icons.
7. **Phone first.** Design and test at 360–430 px. The storefront is a centred column of at most 480 px on larger screens.

8. **Every screen has loading (skeleton), empty and error states.** No blank screens. No raw error text shown to customers.
9. **Build only what `docs/SPEC.md` lists for v1.** Do not add or stub anything from its "Not in v1" list. Do not add a dependency without saying why.
10. **Our own brand only.** Do not copy another company's name, logo, colours, screenshots, icons or product photos.

How to work

- Plan first for anything that touches the database, sign-in, checkout, or more than three files. Wait for approval.
- Keep changes inside the task. Do not refactor unrelated code.
- When done: run typecheck, lint and build, and fix what fails.
- Then append an entry to `docs/PROGRESS.md`: what was built, decisions made, anything left open, and the exact steps a human can follow to verify it.
- Report honestly. If something is unfinished, untested or mocked, say so in the first line of your summary. Never say a test passed unless you ran it and saw it pass.

APPENDIX B: SPEC.MD (PRODUCT SPEC)

[BRAND] — product spec (v1)

Replace `[BRAND]` everywhere once the name is final. Values marked *(setting)* are launch defaults the owner changes in the operator console.

1. What this is

A phone-first web app (installable PWA) for students at three neighbouring campuses in Bhopal to order snacks, drinks and daily essentials from ONE store that holds its own stock. Orders go to a fixed list of hostels and gates in about 15 minutes. One store, one catalogue, one operator team. It is not a marketplace.

2. People

- **Customer** — a student on a phone, usually late in the evening, often on weak hostel Wi-Fi. Wants to order in under a minute.
- **Operator (admin)** — the owner or packer at the store, on a laptop or phone. Must never miss an order and must change stock in seconds.
- **Runner** — uses the same operator order screen on a phone. There is no separate runner app.

3. v1 scope — build exactly this

Storefront

1. **Home** — store status, chosen delivery point and its ETA, search field, category grid, two or three product rails.
2. **Category page** — products of one category, optional subcategory chips, in-stock items first.
3. **Product sheet** — image, name, size, price, MRP, description, manufacturer and country of origin when present, similar items.
4. **Search** — instant, typo-tolerant, alias-aware, never a dead end (section 8).
5. **Cart** — quantity steppers everywhere, floating cart bar, bill breakdown, minimum-order and free-delivery nudges.
6. **Sign-in** — Google, asked only at checkout. One-time profile: name, mobile number, delivery point, room.
7. **Checkout** — address summary, substitution preference, optional note, payment = pay on delivery (UPI or cash).
8. **Orders** — live status timeline, history, order again, cancel while still "placed". Items the store had to remove are shown as unavailable.
9. **Static pages** — About & Contact, Terms, Privacy, Cancellation & Refunds.

Operator console (`/admin`, admins only)

10. **Orders** — live queue with sound, one-tap status changes, remove an item that turns out to be unavailable, call or WhatsApp the customer, mark paid, pack list.
11. **Stock** — instant search, −/+ quantity, one-tap out of stock, low-stock filter.
12. **Products and categories** — add and edit, photo upload with automatic compression, CSV import and export.
13. **Settings** — open/closed, hours, fees, minimum order, support phone, notice text, delivery points.
14. **Today** — orders, revenue (with the split collected by UPI and by cash), average order value, cancellations, top items, orders per hour, failed searches, item requests.
15. **Alerts** — every new order and every customer cancellation is posted to the operators' Telegram group.

4. Not in v1 — do not build or stub

Payment gateway or wallet · coupons and referrals · ratings and reviews · multiple stores or vendors · maps or GPS · runner app · live runner tracking · SMS or OTP · customer push notifications · native apps · scheduled delivery slots · multiple languages · dark mode · staff roles beyond admin.

5. Data model

All money is whole rupees stored as integers. All timestamps are UTC (`timestamptz`) and displayed in Asia/Kolkata.

- **categories** — id, name, slug (unique), image_url, sort_order, is_active.
- **products** — id, sku (unique), category_id, subcategory (optional text), name, brand, size_label (e.g. "70 g"), mrp, price (`price <= mrp`, both `> 0`), stock_qty (`>= 0`), max_per_order (default 5), image_url, keywords (`text[]`: aliases, misspellings, Hinglish), description, manufacturer, country_of_origin (default "India"), rail (`null | late_night | bestsellers | new`), is_active, created_at, updated_at.
- **delivery_points** — id, campus, name, eta_minutes, delivery_fee_override (nullable), sort_order, is_active.
- **profiles** — id (= auth user id), full_name, phone, delivery_point_id, room, created_at, updated_at.
- **admins** — user_id. Rows are added by hand in the Supabase dashboard. No client can ever write to this table.
- **settings** — a single row: store_open (manual switch), opens_at, closes_at (local time; closing may be after midnight), min_order_value, delivery_fee, free_delivery_above, max_active_orders_per_user, support_phone, notice_text.

- **orders** — id, order_no (short human number from a sequence starting at 1001), client_request_id (uuid, unique), user_id, status, delivery_point_id, room, customer_name and customer_phone (snapshots), items_total, delivery_fee, grand_total, payment_method (`pay_on_delivery`), payment_status (`unpaid | paid`), paid_via (`upi | cash`, nullable), substitution_pref (`call_me | replace_similar | remove_item`), note, cancel_reason, placed_at, accepted_at, packed_at, out_at, delivered_at, cancelled_at.
- **order_items** — id, order_id, product_id, name_snapshot, size_snapshot, unit_price, mrp, qty, line_total, removed (boolean, default false).
- **search_misses** — id, query (at most 80 characters), user_id (nullable), created_at.
- **item_requests** — id, text (at most 120 characters), user_id (nullable), created_at.
- **client_errors** — id, message (at most 500 characters), route, user_agent, user_id (nullable), created_at.

The last three tables are insert-only for clients, signed in or not, because most searching happens before sign-in. No client can read them.

Launch defaults *(setting)*: hours 18:00–01:00 · minimum order ₹99 · delivery fee ₹15 · free delivery above ₹299 · at most 2 active orders per user.

6. Order rules — enforced in the database, never in the browser

Orders are created only by the database function `place_order`. It runs as one transaction and must:

1. Require a signed-in, non-anonymous user with a complete profile.
2. Reject if the store is closed: `store_open` is false, or the current Asia/Kolkata time is outside opens_at–closes_at. Handle closing after midnight.
3. Require an active delivery point.
4. For every item: the product is active, qty is between 1 and max_per_order, and stock is sufficient.
5. Read prices from the products table at that moment. Any price sent by the client is ignored.
6. Require `items_total >= min_order_value`.
7. Set delivery_fee to the point's override or the default, and to 0 when `items_total >= free_delivery_above`.
8. Reject if the user already has max_active_orders_per_user orders that are neither delivered nor cancelled.
9. Decrement stock for every item in the same transaction. Two simultaneous orders for the last unit: exactly one succeeds. No deadlocks.
10. Be idempotent: a second call with the same client_request_id from the same user returns the first order and changes nothing.
11. On failure change nothing, and return a machine-readable reason plus, for stock or price problems, the current price and available quantity of each affected item so the cart can correct itself.

7. Status flow

`placed → accepted → packed → out_for_delivery → delivered`

- `cancelled` is reachable from placed, accepted and packed by an admin (reason required), and from placed by the customer.
- Cancelling restores the stock of every item exactly once.
- Each transition stamps its time. No other transitions are allowed, by anyone.
- Before an order is packed, an admin can remove an item that turns out to be unavailable. The line is marked removed, not deleted. Totals are recalculated, the delivery fee never goes up because of a removal, and that product's stock is set to 0. Removing the last remaining item cancels the order.
- Payment is marked separately by an admin: paid, via UPI or cash.

8. Search — must never dead-end

- The whole active catalogue is loaded once and searched in the browser. Results appear as the user types, with no network call per keystroke.
- Indexed fields, highest weight first: name, brand, keywords, subcategory, category, size_label.
- Tolerates prefixes ("mag"), typos ("magi", "biskit", "choclate") and aliases stored in keywords ("doodh", "namkeen", "cold drink").
- Result order: (1) in-stock exact and prefix matches, (2) in-stock fuzzy matches, (3) out-of-stock matches, dimmed, each followed by a "Similar in stock" row.
- No match: say so plainly, then show bestsellers from the nearest category, then a one-tap "Request this item" that writes to item_requests.
- Log a query to search_misses only when it has three or more characters, returned nothing, and the user paused 1.5 s or left the screen. Never per keystroke.
- Empty search screen: recent searches (stored on the device, at most 6, clearable), popular searches, category shortcuts.

9. Catalogue loading and similar items

- One request loads active categories and products. It is cached and persisted on the device so repeat visits render instantly, then refreshed on focus and every 60 seconds.
- After each refresh the cart is reconciled: items that went out of stock, were deactivated or changed price are corrected, and the user is told once.
- Similar items for a product: in stock, same subcategory first, then same category, nearest in price, excluding itself, at most 6.

10. Operator console details

- **New-order alert** — after one tap on an "I'm online" switch, a repeating chime plays until the order is acknowledged, the tab title flashes, and the screen is kept awake. A visible warning appears if sound is blocked or the live connection drops. It reconnects automatically.
- **Order card** — order number, minutes since placed (amber after 5, red after 10), delivery point and room, name, phone (tap to call), items and quantities, total, payment status, substitution preference, note.
- Every admin action goes through a database function or an admin-only policy. Hiding a button is not security.
- "Today" means the business day 06:00–06:00 Asia/Kolkata, because the store trades past midnight.
- CSV import follows `docs/catalog-template.csv`, validates every row before writing anything, and upserts by sku.

11. Security

- Row Level Security on every table.
- Anyone, signed in or not, can read: active categories, active products, active delivery points, settings.
- Anyone, signed in or not, can insert into search_misses, item_requests and client_errors, within the length limits. Nobody but an admin can read them.
- A signed-in user can read and update only their own profile, and read only their own orders and order items.
- No client inserts, updates or deletes orders or order_items directly. Only `place_order`, `cancel_my_order` and the admin functions do.
- Admins (members of the admins table) can read everything and write products, categories, delivery points, settings and order status.
- The admins table has no client-side insert, update or delete policy at all.
- Product images: public read, admin-only upload.
- The Supabase secret key never appears in the client bundle or the repository.

12. Non-functional

- **Phone first** — designed and tested at 360–430 px. The storefront stays a centred column of at most 480 px on larger screens. The operator console is responsive for phone and laptop.
- **Fast on hostel Wi-Fi and 4G** — home is usable within about 2.5 seconds on a mid-range Android phone; images are lazy-loaded with reserved space; customers never download the operator console code.
- **Resilient** — the cart survives reloads; every request has a clear retry; placing the same order twice by accident is impossible (section 6, rule 10).
- **Accessible basics** — 44 px tap targets, visible focus, 4.5:1 text contrast, reduced motion respected.

- **Language** — English interface. Product keywords may include Hinglish.

13. Legal display

- Every product shows its MRP and net quantity. The selling price can never exceed MRP (database constraint).
- The product sheet shows manufacturer and country of origin when the owner has entered them.
- Footer pages carry the business name, address, support phone and food-licence number once the owner supplies them.
- Nothing from another company's brand is used: no names, logos, colours, screenshots, icons or product photos.

14. Owner to fill before launch

Brand name · delivery points per campus · opening hours · fees and minimum order · support phone · business details for the footer pages · the real catalogue and photos.

APPENDIX C: DESIGN.MD (DESIGN SYSTEM)

[BRAND] — design system (v1)

One look, used everywhere. If a screen needs something that is not here, add it here first, then use it.

Principles

1. **The product photo is the hero.** White cards, quiet chrome, one accent colour.
2. **Thumb first.** Everything a customer needs is reachable one-handed at 360–430 px.
3. **Nothing jumps.** Space is reserved for every image and every value that loads later. Skeletons, not spinners.
4. **Say it once.** One typeface, one accent, one radius family, one shadow.

Tokens

Colour

Token	Value	Use
`--brand`	`#2451E6`	Primary actions, links, active states, cart bar
`--brand-pressed`	`#1B3FC0`	Pressed state
`--brand-soft`	`#EEF2FF`	Tinted background behind brand-coloured text
`--ink`	`#0F172A`	Headings and body text
`--ink-2`	`#475569`	Secondary text, MRP, captions, "Out of stock"
`--ink-3`	`#94A3B8`	Placeholders and disabled controls only. Never for information
`--surface`	`#FFFFFF`	Cards, sheets
`--canvas`	`#F5F6F8`	Page background
`--line`	`#E6E8EC`	Borders, dividers
`--success`	`#15803D`	Delivered, savings
`--warning`	`#B45309`	Low stock, closing soon, an order waiting too long
`--danger`	`#DC2626`	Errors, destructive actions

Category tile tints, used in order and repeated: `#FFF4E5`, `#EAF4FF`, `#EEF9F0`, `#FDEFF2`, `#F3EEFF`, `#FFF9DB`, `#E8F7F7`, `#F4F1EC`.

Rules: the brand colour appears only on things you can tap, and on the cart bar. Body text is never brand-coloured. "Out of stock" is neutral, not red. The brand colour is the owner's to change: one token, and the whole app follows.

Type

Plus Jakarta Sans, variable, self-hosted (no third-party font request). Fallback: `system-ui`. Prices and quantities use tabular numbers.

Style	Size / line height	Weight	Use
display	22 / 28	700	Screen titles
title	18 / 24	700	Section titles, header ETA
subtitle	16 / 22	600	Prices, buttons
body	14 / 20	500	Product names, text
caption	12 / 16	500	Size, MRP, hints
label	11 / 14	700, uppercase, +4% tracking	Badges only

Nothing a customer must read is smaller than 12 px.

Space, shape, depth, motion

- **Space** — 4-point scale: 4, 8, 12, 16, 20, 24, 32. Screen gutter 16. Card padding 12. Grid gap 12.
- **Radius** — card 14, button and input 12, image 10, chip and pill 999.
- **Depth** — cards have a 1 px `--line` border and no shadow. Only floating things (cart bar, sheets, toasts) cast one shadow: `0 8px 24px rgba(15, 23, 42, 0.12)`.
- **Motion** — 160 ms ease-out for state changes, 240 ms for sheets. No bounce, no parallax. Honour `prefers-reduced-motion`.
- **Icons** — lucide-react, 20 px, stroke 1.75. No emoji anywhere in the interface.

Layout

- Storefront: single column, max width 480 px, centred on `--canvas` on larger screens.
- Product grid: 2 columns from 360 px. Category grid: 4 columns.
- Fixed elements respect the device safe areas. The cart bar floats above the bottom inset.
- Operator console: cards on phones, a two-pane layout from 900 px.

Components

- **AppHeader** — line 1: "Delivery in 12 min" (title) with a status dot. Line 2: the delivery point with a chevron, tap to change. Closed state: "Closed · Opens 6:00 pm". Account icon on the right.
- **SearchField** — 48 px high, `--canvas` fill, search icon, placeholder "Search snacks, drinks, essentials".
- **CategoryTile** — square, tinted background, illustration on top, name below (caption, weight 600), at most 2 lines, centred.
- **ProductCard** — square image area (white, image contained with an 8 px inset); size_label (caption, `--ink-2`); name (body, weight 600, clamped to 2 lines with a fixed height so cards in a row align); price (subtitle, 700) with MRP struck through (caption, `--ink-2`) only when MRP is higher; AddStepper bottom right. Badge top left, "10% OFF" in label style on `--brand-soft`, only when the discount is 5% or more.

- States: default · in cart · low stock ("Only 2 left", caption in `--warning`, when stock is 3 or less) · out of stock (image at 50% opacity, neutral "Out of stock" pill, no add button, "See similar" link) · no image (neutral placeholder with the category icon) · loading (skeleton of identical size).
- **AddStepper** — default: outlined "ADD" button (`--brand` border and text, 36 px high, at least 72 px wide, tap area padded to 44 px). In cart: solid `--brand` stepper "− 2 +" with the same footprint, so nothing shifts. At the stock or per-order cap the plus is disabled and a toast explains why.
- **PriceTag** — "₹45": rupee sign, no space, no decimals.
- **ProductRail** — title with "See all"; horizontal scroll with snap; cards 140 px wide.
- **CartBar** — floating, `--brand` fill, white text: "3 items · ₹245" on the left, "View cart" with a chevron on the right. Slides in when the cart has items.
- **BillSummary** — rows for item total, delivery fee (or "Free") and total; a savings line in `--success` when price is below MRP; a nudge line such as "Add ₹40 more for free delivery".
- **BottomSheet** — drag handle, at most 90% of the viewport height, closes on swipe down and on backdrop tap.
- **StatusTimeline** — vertical: Placed, Accepted, Packed, Out for delivery, Delivered, each with its time once reached; the current step in `--brand`.
- **Button** — primary (solid `--brand`), secondary (outline), ghost. 48 px high. One primary per screen.
- **Chip** — pill filter. Selected: `--brand-soft` fill with `--brand` text.
- **Toast** — one line, at the bottom, above the cart bar, dismisses after 2.5 s.
- **EmptyState** — icon, one sentence, one action. **ErrorState** — one sentence and "Try again".
- **Skeletons** — card, tile, row and bill variants matching the real component sizes exactly.

Imagery

- Product photos: square, plain white background, product filling about 80% of the frame, shot straight on. Stored as WebP, 600 px, under about 40 KB.
- Category tiles: one illustration style for all of them, on the tint colours above.
- Logo: a wordmark in the display style plus one simple original glyph. Placeholder until the owner supplies a final mark.
- Our own photos and illustrations only.

Copy

Sentence case. Short. No exclamation marks, no placeholder text, no invented numbers. Say what happened and what to do next: "Maggi Masala is out of stock. We removed it from your cart."

Quality bar — check every customer screen against this

1. Nothing shifts when images or data arrive.
2. Every tappable thing is at least 44 × 44 px.
3. Product names never overflow, and card bottoms align across a row.
4. Prices follow the PriceTag format everywhere.
5. Works at 360 px with no horizontal scroll.
6. Loading, empty and error states exist and look designed.
7. Text contrast is at least 4.5:1, and focus is visible.
8. Only tokens from this file are used. No stray colours, sizes or shadows.